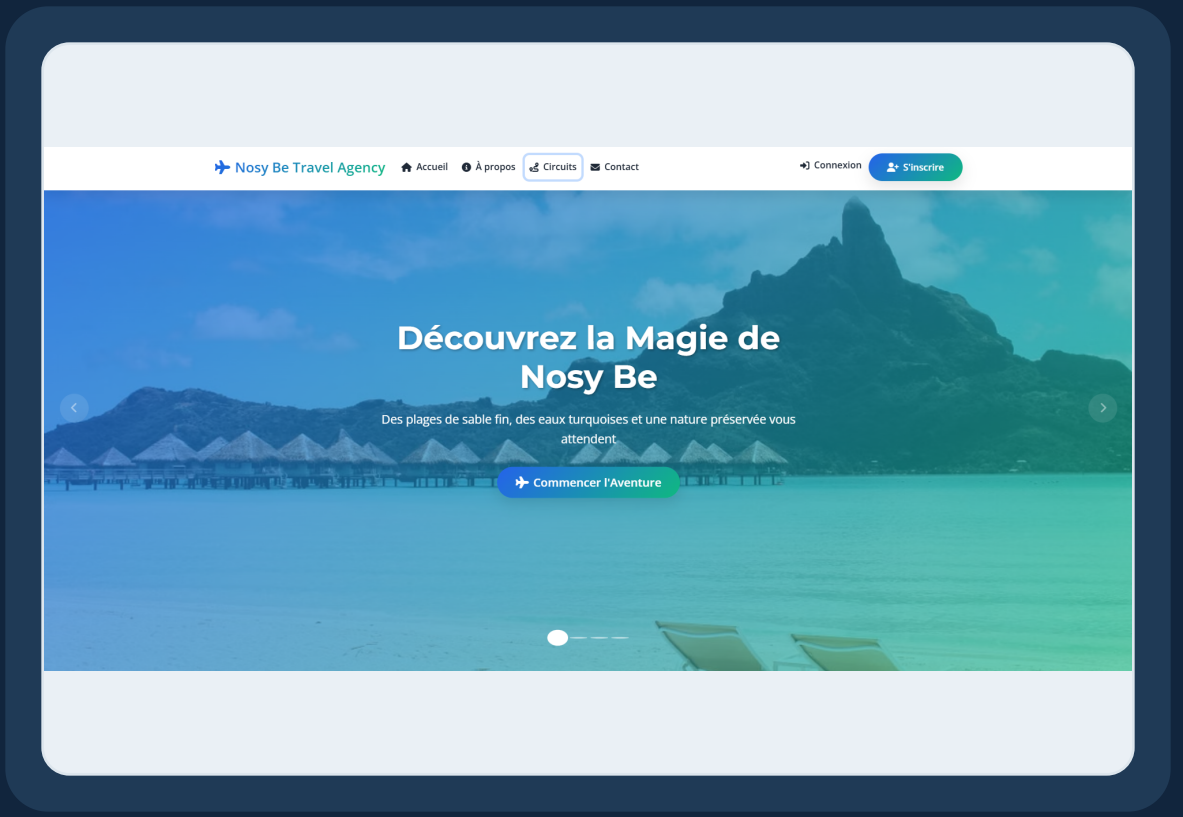
TECHNICAL CASE STUDY · ENGLISH EDITION

# LALANA

Nosy Be Travel Agency · Tourism management platform.




## WEB APPLICATION · CASE STUDY

Tourism discovery, reservations and operational management.

## TECH STACK

Java 17 · Spring Boot 3.4.4 · Thymeleaf · PostgreSQL

Presented by: RANDRIAMANANTENA Fitahiana Lafatra

# Project overview

A web solution organized into three business areas.

### CONTEXT · TRAVEL AND BOOKING

Tours, destinations, flights and bookings within the same system.

### OBJECTIVE · CENTRALIZE JOURNEYS

Discover an offering, check availability and track bookings.

### USERS · THREE AREAS

Customer, Manager and Administrator with different user journeys.

### FOUNDATION · SPRING MVC + POSTGRESQL

Thymeleaf pages, business services, JPA repositories and Spring Security roles.

# From tourism catalog to operational tracking

A coherent interface for travelers and management staff.

### 01 · DISCOVERY

Present tours, flights and destinations with clear information.

### 02 · BOOKING

Manage selected offers, the number of travelers and availability.

### 03 · OPERATIONS

Allow teams to oversee cases and indicators.

APPROACH · One Java platform with interfaces tailored to each type of user.

# Three profiles, distinct responsibilities

Access control is designed to be enforced on the server.

### VISITOR / CUSTOMER · EXPLORE AND BOOK

Browse offerings, create an account, consult requests and track bookings.

### MANAGER · PROCESS REQUESTS

Manage tours, flights, customers and booking requests within granted permissions.

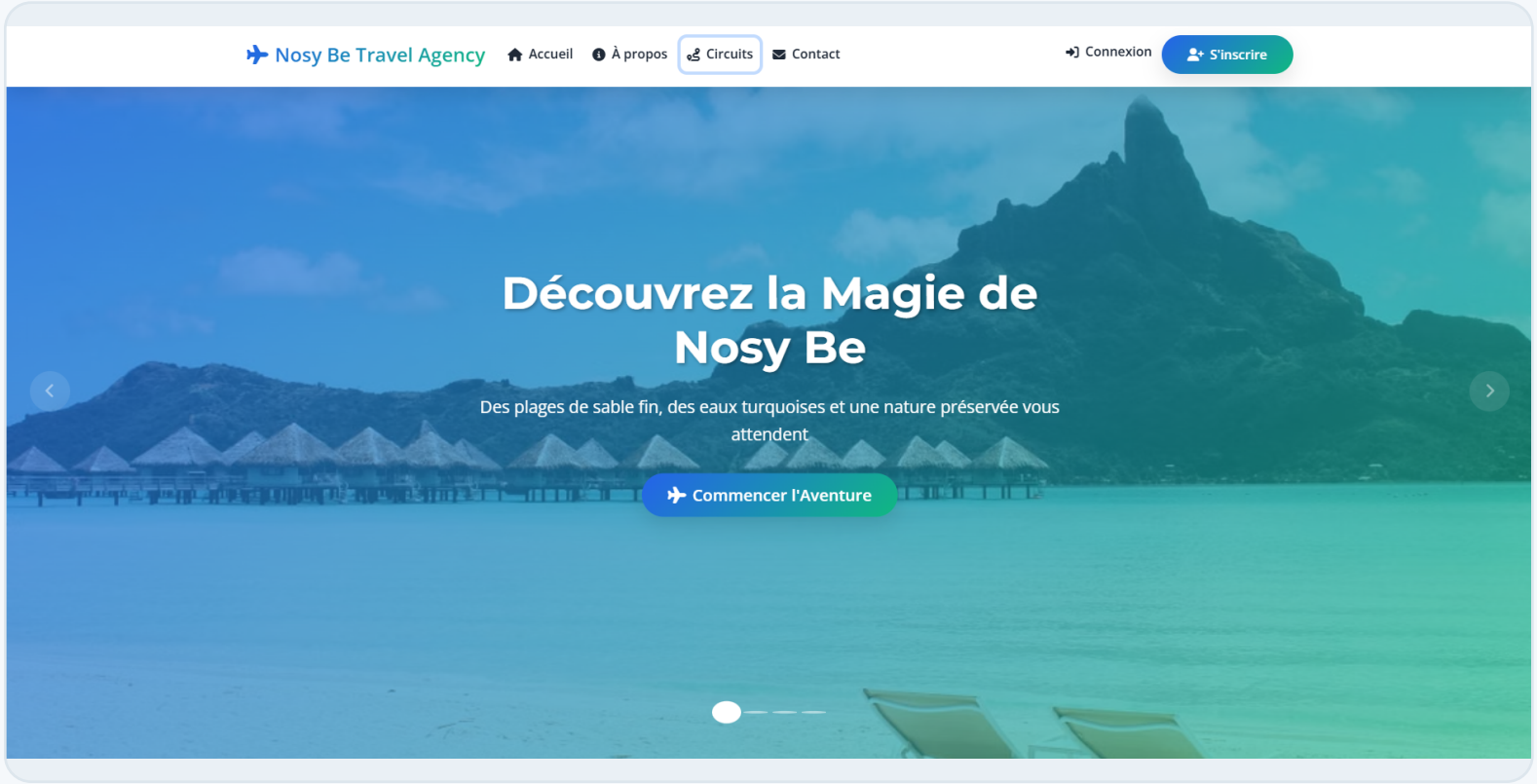
### ADMINISTRATOR · SUPERVISE

Oversee users, tours, destinations, flights, bookings, statistics and activity.

PRINCIPLE · Data access requires server-side checks, not merely hidden navigation links.

# Home page: discover Nosy Be

Public view - navigation, carousel and call to explore the tours.



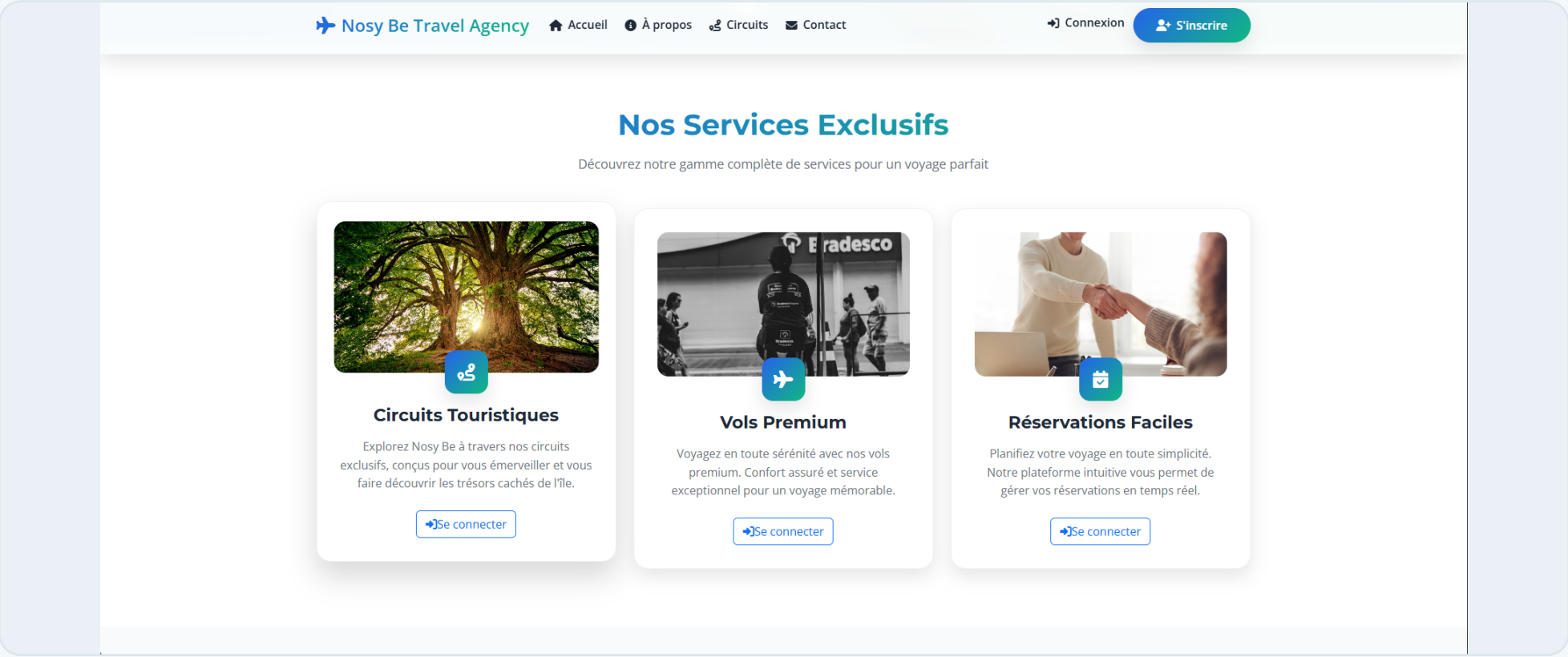

Supplied project screenshot; it does not by itself prove public deployment.

## WHAT THE SCREENSHOT SHOWS

Navigation to tours and contact; immersive presentation of the destination; access to sign-up and login.

# Service presentation

Offerings are grouped into tourist tours, flights and bookings.




Public interface: three categories displayed in one grid.

# A visible value proposition

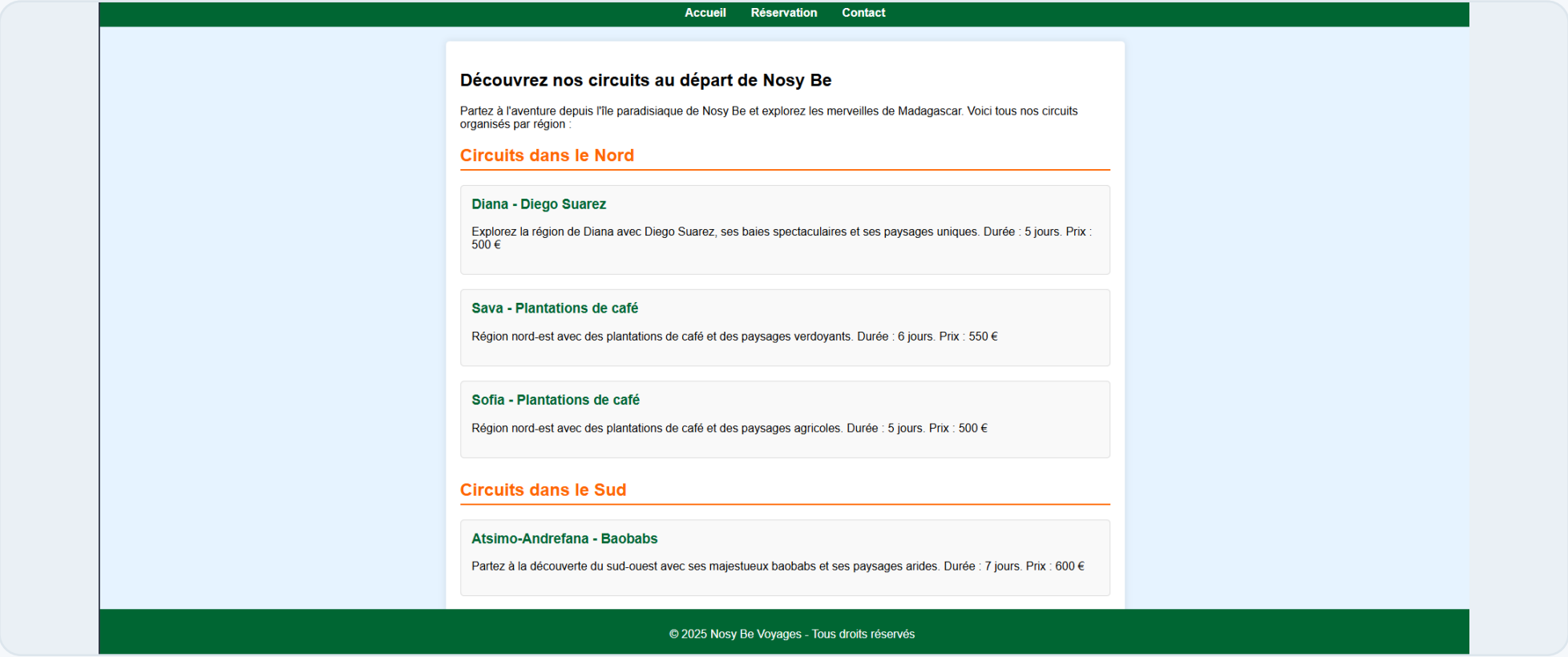
Six arguments displayed in the “Why choose us?” section.




These statements are interface content, not independently verified contractual guarantees.

# Tourist tour catalog

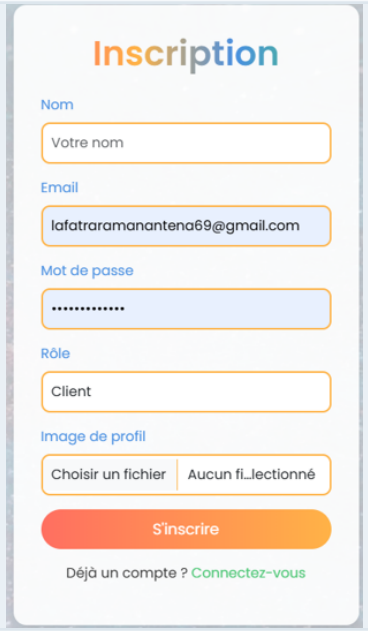
Example page presenting several itineraries and their details.




Examples shown: regions, durations and indicative prices - screenshot only.

# Registration and login

Forms from the project for account setup and authenticated access.







## REGISTRATION

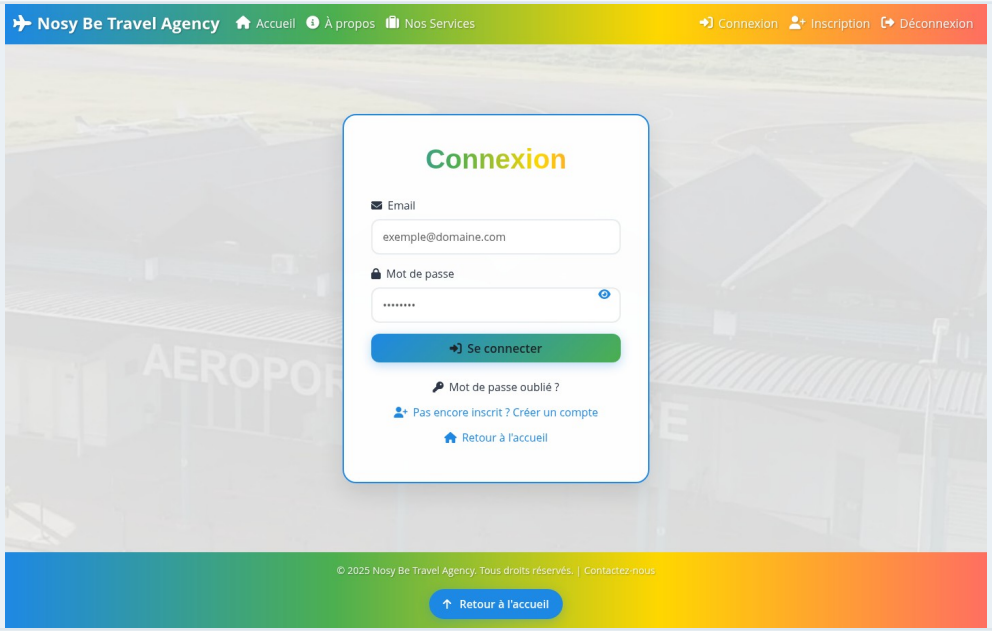
User account fields and proposed role.

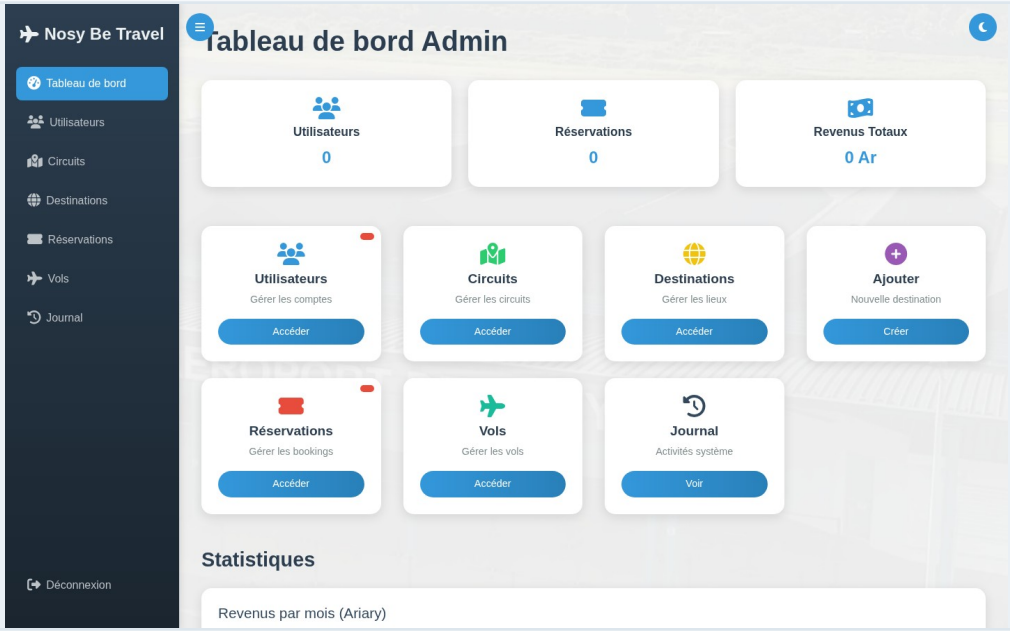
## LOGIN

Secured access to the private area.

# Login and dashboards

Access to private spaces and administrator overview.




## ACCOUNT ACCESS

Authentication form and navigation to private spaces.

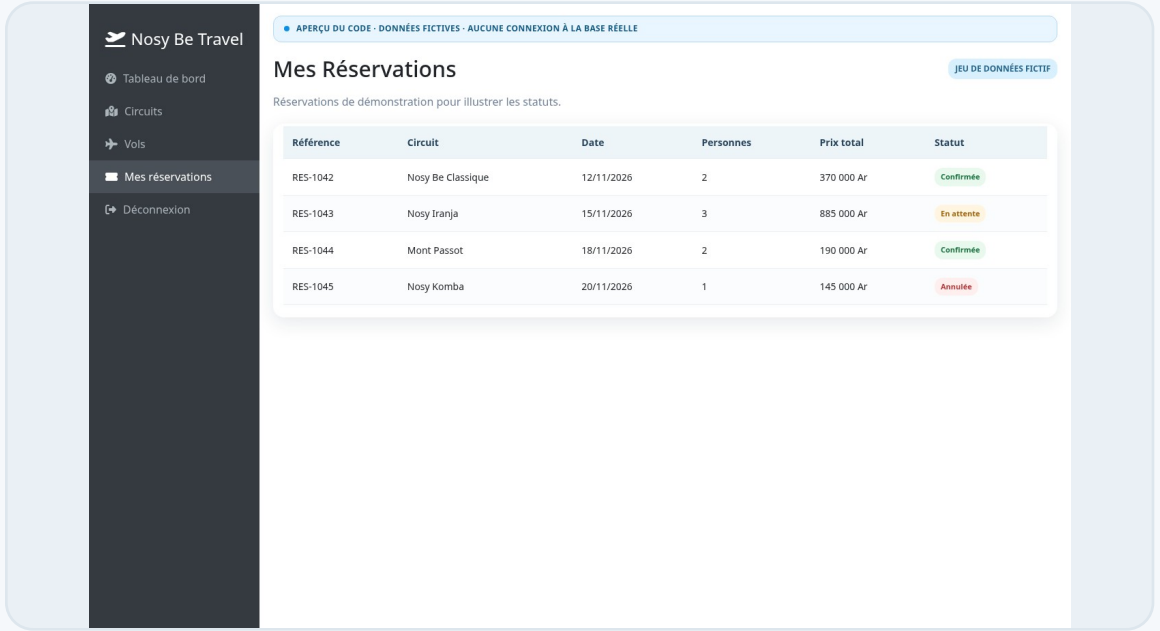
## ADMINISTRATOR SPACE

ADMIN view: KPIs, access to management modules and a statistics section.

# Manager and booking tracking

Dedicated operational and customer views.




## MANAGER

Dashboard for management operations and indicators.

## CUSTOMER

Customer view: booking history and statuses.

# Spring MVC with server-side rendering

Layered architecture - this project does not use React.

### 01 · BROWSER

HTML / CSS / JavaScript. Thymeleaf views and Chart.js charts.

### 02 · ROUTING

MVC controllers. Forms, routes and view models.

### 03 · BUSINESS LOGIC

Java services. Booking rules and workflow coordination.

### 04 · DATA

JPA / PostgreSQL. Repositories, entities and SQL relations.

CROSS-CUTTING · Spring Security (access), JavaMailSender (emails), WebSocket/STOMP (chat).

# Main entities in the business domain

Key entities and business links in the documented model.

### USER

Accounts, roles and identities.

### TOUR

Tourist offers, price and capacity.

### DESTINATION

Places and descriptions.

### FLIGHT

Routes and availability.

### BOOKING

Customer, offer, travelers and status.

### PAYMENT

Associated amount and status.

RELATIONSHIPS · User → Booking → Tour / Flight; Booking → Payment; Tour ↔ Destination. Other documented items: tour_destination association, activity log and message templates.

# Booking journey

Tour only, flight only or package: workflows described in the code.

### 01 · DISCOVER

Browse tours, flights and destinations.

### 02 · CHOOSE

Select an offer and the number of travelers.

### 03 · CHECK

Check capacities and compatibility.

### 04 · TRACK

Register the request and its status.

IMPORTANT · Resistance to simultaneous bookings was not validated by supplied end-to-end acceptance tests.

# Statistics planned in the dashboards

Chart.js components are present in the templates.

### 01 · MONTHLY REVENUE

Monthly aggregation of booking amounts.

### 02 · BOOKINGS BY STATUS

Counts and distribution of business statuses.

### 03 · SUMMARY INDICATORS

Users, bookings and operational information.

INTERPRETATION · A booking total is not, by itself, evidence of collected payments.

# Security, validation and future improvements

A readable architecture, with controls still to be completed.

### DOCUMENTED · AUTHENTICATION AND ROLES

Spring Security, BCrypt password encoding and account services.

### TO CONSOLIDATE · WORKFLOW VALIDATION

SQL and integration tests, concurrent bookings, email handling and responsive design.

### EVOLUTION · OPERATIONAL ROBUSTNESS

Strengthen security configuration, processing and observability.

SCOPE · Sources and interfaces reviewed; production deployment and complete acceptance testing are not established.

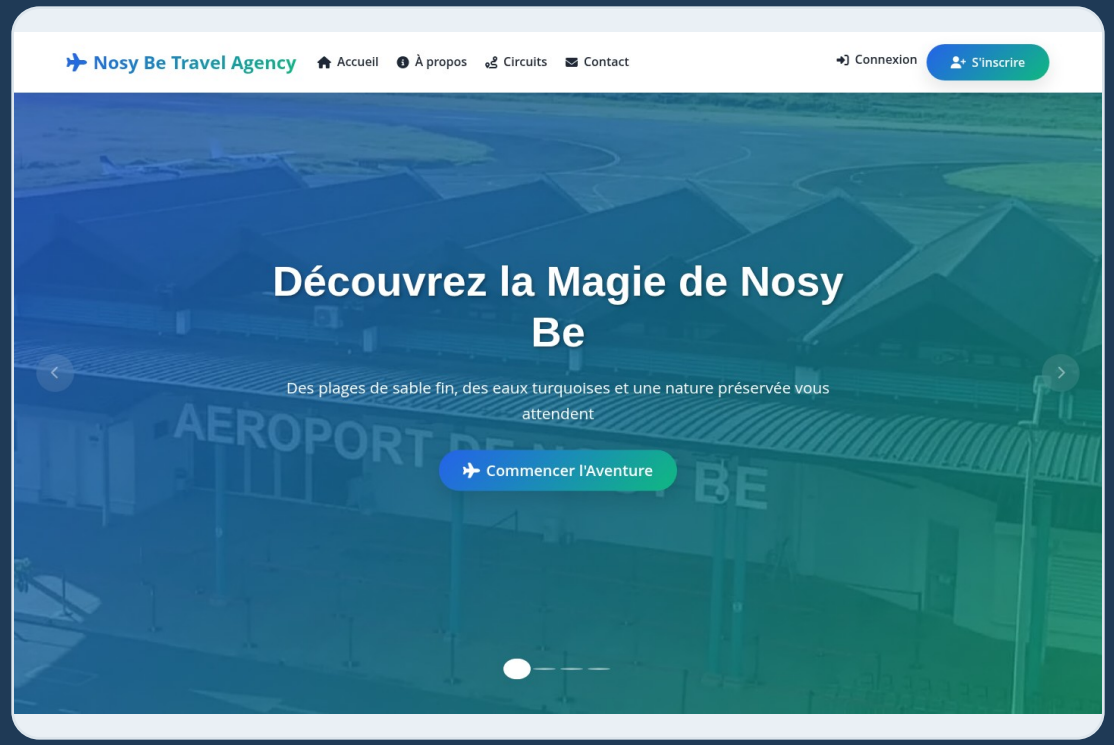
TECHNICAL CASE STUDY · ENGLISH EDITION

# LALANA

A Java web architecture case study focused on bookings and relational persistence.

## CORE TOPICS

Spring MVC · JPA / PostgreSQL · Role management · Business rules




Screenshots and architecture based on project materials. No production deployment is assumed.

Presented by: RANDRIAMANANTENA Fitahiana Lafatra